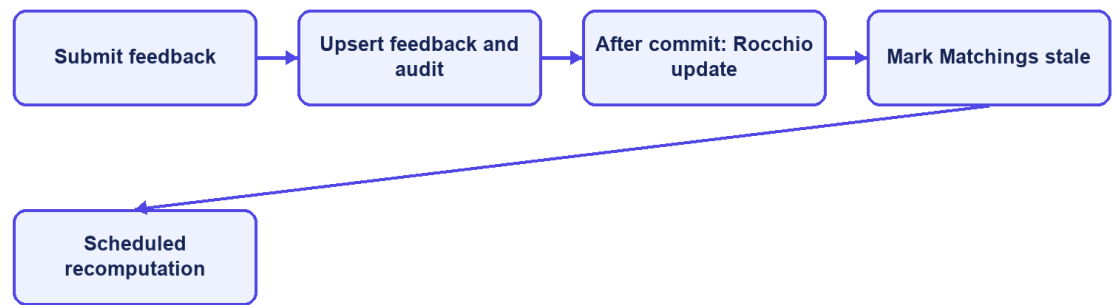


Feedback recomputation sequence



CareerFit IT AutoPilot — thesis diagram

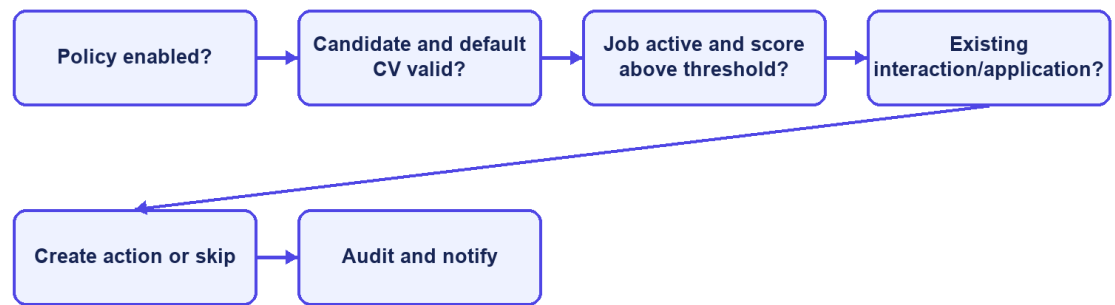
Figure 3.4. Feedback learning and recomputation sequence

3.5.5 AutoFit Executes a Policy-Governed Action

Automation begins with a persisted policy associated with a user. Candidate-oriented policy data includes enablement, score thresholds, digest and high-match notification preferences, daily limits, cooldown, quiet hours, timezone, pause state, and auto-apply options. Scheduled scans resolve the user's Candidate and default CV, require a successfully scored CV, select eligible matches, and apply policy and state checks before sending a notification or creating an application. The run-now operation provides a controlled testing path without changing the policy rules.

The scheduler contains five concrete tasks: stale Matching recomputation every 30 minutes; daily digest at 08:00 Asia/Ho_Chi_Minh; expired email-action cleanup at 03:00; high-match notification scans every four hours with an additional 07:00–22:00 ICT window; and auto-apply scans every two hours. The annotation expressions read their delays, cron expressions, and time zone from `app.scheduler.*` properties, providing one environment-configurable scheduling source.

AutoFit decision flow



CareerFit IT AutoPilot — thesis diagram

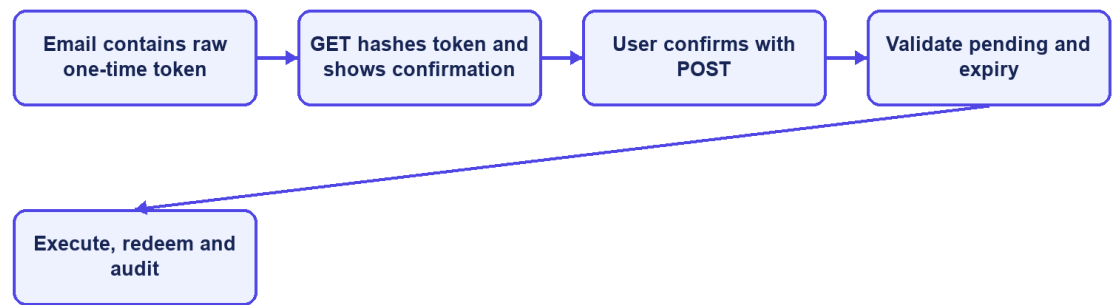
Figure 3.5. AutoFit decision flow

3.5.6 Email Feedback Action

For match notifications and digests, the backend creates EmailAction records with a high-entropy raw token for the recipient, a persisted SHA-256 token hash, and a 72-hour expiry. The public `/api/email-action/redeem` endpoint validates the hash, pending state, and expiry. Reuse is rejected, expired pending actions are marked expired, and a scheduled task later removes old records.

The email-action design uses two steps. GET hashes and validates the supplied high-entropy token and displays a confirmation page without changing recruitment state. A deliberate POST repeats validation, executes the supported action, marks the token as redeemed, and records the outcome. Only the SHA-256 token hash is persisted, while expiry and status checks prevent normal replay. Automated link scanners therefore cannot execute an action by merely visiting the GET URL.

Hardened email-action sequence



CareerFit IT AutoPilot — thesis diagram

Figure 3.6. Secure email-action confirmation and redemption sequence

3.6 System Architecture

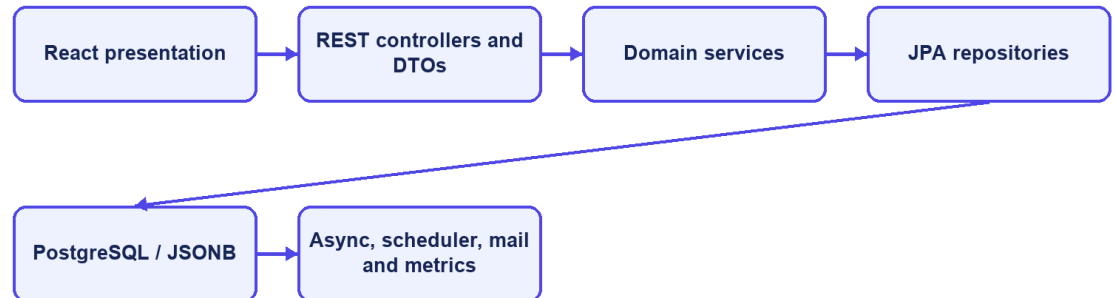
CareerFit is implemented as a modular monolith. The frontend is a separate React application, while the backend is one Spring Boot deployable application organized into domain packages. This is not a microservice architecture: modules share one process, one primary PostgreSQL database, common security and exception handling, and direct in-process service calls. The modular monolith reduces deployment and distributed-transaction complexity while preserving boundaries that can support later extraction if scale or ownership requires it.

The presentation layer contains React pages, reusable components, route guards, API mapping, and query state. It calls the backend over HTTP and must not be trusted to enforce protected business rules. The API layer contains controllers and DTO validation. The service layer owns use-case orchestration, transactions, scoring, policy decisions, and ownership checks. Repositories map domain entities to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database

stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

Modular-monolith architecture



CareerFit IT AutoPilot — thesis diagram

Figure 3.7. CareerFit container and component architecture

3.7 Module Design

Table 3.4. Backend modules and responsibilities

Module	Main responsibility	Representative services
Auth and Security	Registration, login, passwordless flow, JWT validation, account resolution, role enforcement	AuthService, JwtService, security filters
Candidate and CV	Candidate profile, portfolio, CV lifecycle, storage, extraction/OCR, validation	CandidateProfileService, CvManagementService, CvIngestionService, PdfExtractionService
Job and Employer	Public job search, recruiter-owned job lifecycle, employer pages and profile	JobService, EmployerService
Matching	TF-IDF/cosine scoring, labels, reasons, batch matching, candidate cards and recruiter ranking	TfidfService, ScoringService, MatchingService, MatchingBatchService, MatchingQueryService
Recommendation	Candidate-profile job ordering and similar-job retrieval	RecommendationService
Application	Manual applications, withdrawal, applicants, invitation, recruiter status transitions	ApplicationService
Feedback	Feedback validation, persistence, Rocchio update, recomputation signaling	FeedbackService, RocchioService

Module	Main responsibility	Representative services
Automation	Policy persistence, run-now, auto-apply, pause/resume, scheduled orchestration	AutomationPolicyService, AutoApplyService, AutomationScheduler
Notification	Policy guard, delivery log, SMTP/no-op sending, actionable email	NotificationPolicyGuard, NotificationEmailService, EmailActionService
Analytics and Admin	Market/role analytics, events, administrative dashboards, moderation and monitoring	AnalyticsService, AdvancedAnalyticsService, administrative services
Audit and Common	Audit persistence, shared responses, exceptions, validation utilities	Audit repository/service and common components

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

3.8 Data Design

3.8.1 Core Identity and Recruitment Data

`user_account` stores identity, role, activation, verification, language, and password hash. A Candidate account owns one candidate profile, multiple CVs, portfolio links, and portfolio project records. A Recruiter account owns an `employer_profile` and jobs through `recruiter_id`. `job` stores the original JD, structured recruitment attributes, salary mode, language, status, source provenance, and vector data.

`matching` is the unique association between one CV and one job. It stores raw and normalized scores, label, potential metadata, reasons, and the recomputation marker. `application` links Candidate and Job and may reference the CV and Matching used at application time. `feedback` links a Matching with its actor and source channel. `recommendation_interaction` records behavioral events such as viewed, skipped, applied, saved, not interested, or show similar.

3.8.2 Automation, Communication, and Operational Data

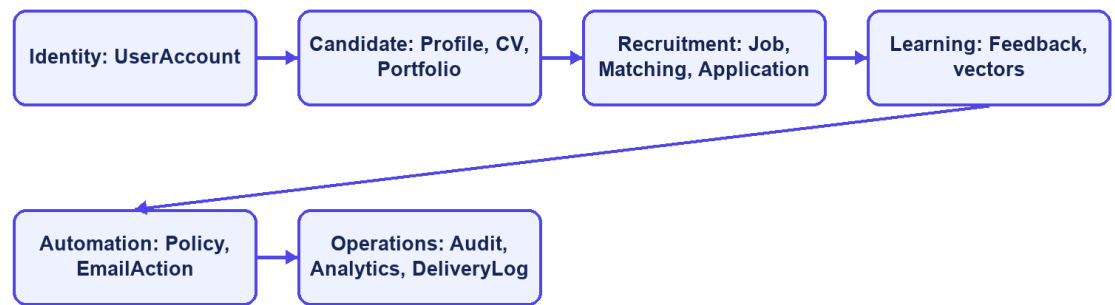
automation_policy stores one policy per user. email_action and email_token currently represent two distinct tokenized workflows. Notification delivery records support deduplication and quota/cooldown decisions. audit_log stores actor, action, target, result, source channel, request metadata, and structured metadata. Job trend, market snapshot, and analytics event tables support aggregate dashboards without changing the transactional meaning of matching records.

Table 3.5. Key integrity constraints

Constraint	Purpose
Unique normalized user email	Prevent duplicate identities that differ only by case
One Candidate per user and one policy per user	Preserve one-to-one domain ownership
Partial unique default CV per Candidate	Ensure at most one active default CV
Unique Matching per CV and Job	Prevent duplicate scores for the same pair
Unique Application per Candidate and Job	Prevent duplicate applications
Unique feedback per Matching and actor	Prevent ambiguous duplicate current judgments
Check constraints on roles, statuses, labels, salary mode, and source channels	Reject invalid domain states at the database boundary
Optimistic version columns on mutable core entities	Detect conflicting concurrent updates
Source hash index on imported jobs	Support idempotent scraped-job upsert

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application validates entity-to-schema compatibility instead of silently generating a different database. Indexes support common access paths such as active jobs, recruiter jobs, CV ownership, matching by score, applications by candidate or job, feedback history, tokens, and audit chronology.

Data domains and relationships



CareerFit IT AutoPilot — thesis diagram

Figure 3.8. CareerFit logical entity relationships

3.9 Security, Failure, and Consistency Design

3.9.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

3.9.2 Failure Handling

CV processing represents long-running work with statuses rather than holding a browser request open until every score completes. Extraction or validation failure records a failure state and reason. CV and Job matching tasks are submitted through an after-commit executor so background work cannot read uncommitted rows. Batch scoring isolates individual pair failures, and scheduled recomputation keeps unsuccessful rows marked for later review instead of falsely marking them complete.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

3.9.3 Identified Design Risks

Table 3.6. Design risks and required treatment

Risk	Current control	Remaining treatment
Unauthorized cross-account access	URL role rules and service ownership checks	Maintain negative integration tests for every identifier-based operation
Duplicate applications or matchings	Service checks and unique constraints	Convert database conflicts into stable API errors
Replayed email action	Pending/redeemed state and expiry	Implemented hashed storage, GET confirmation, and POST execution; retain replay and expiry tests
Link scanner triggers action	GET is non-mutating and displays confirmation	Add rate limiting and deployment-specific origin controls
Scheduler/configuration drift	Scheduler timing reads from app.scheduler properties	Maintain configuration and schedule-boundary tests
Sensitive data in logs	Structured audit metadata and DTO boundaries	Review retention, redaction, access, and exported evidence

Risk	Current control	Remaining treatment
Lexical scoring bias or omission	Reasons, validation, feedback, and HITL	Evaluate representative data; never treat score as hiring probability
Local file-storage loss or exposure	Configured path and Docker volume	Define backup, encryption, malware scanning, and retention before production

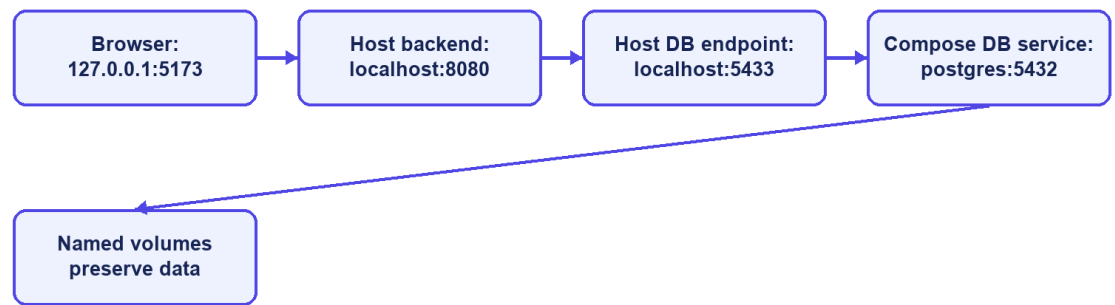
3.10 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, depends on PostgreSQL health, and mounts a named volume for CV storage. The React/Vite frontend is normally run separately on port 5173 during development and calls the backend API. Environment variables override database credentials, JWT secret, application base URL, CORS origins, mail settings, OCR executable and languages, and storage path. Tesseract with Vietnamese and English language data is included in the backend container path described by the project Docker configuration.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

Local deployment addresses



CareerFit IT AutoPilot — thesis diagram

Figure 3.9. Local and containerized deployment topology

3.11 Chapter Summary

This chapter defined the actors, requirements, use cases, architecture, data model, security boundaries, failure handling, and deployment design. The main design rule is to keep matching evidence, application state, user policy, and automated actions separate. Chapter 4 explains how these decisions are implemented.